Oto kompleksowa ściągawka i przewodnik po podstawach skryptowości w języku **Bash**, przygotowana w czytelnej formie idealnej do publikacji na portalu edukacyjnym:

Ściągawka: Podstawy powłoki Bash i tworzenia skryptów

1. Pierwszy skrypt i Shebang

- Każdy skrypt w systemach uniksowych rozpoczyna się od tzw. **shebanga** (#!/bin/bash), który informuje system, jaki interpreter ma zostać użyty do wykonania pliku.
- Znak # (hasz) oznacza komentarz – wszystko, co znajduje się za nim w tej samej linii, jest pomijane przez interpreter.
- Aby plik ze skryptem stał się plikiem wykonywalnym, należy nadać mu odpowiednie uprawnienia za pomocą polecenia chmod +x nazwa_skryptu.
- Uruchomienie skryptu z katalogu bieżącego (jeśli jego ścieżka nie jest dodana do zmiennej PATH) wymaga użycia przedrostka ./, np. ./skrypt.

2. Polecenie echo i obsługa strumieni

- Polecenie echo służy do wyświetlania komunikatów na standardowym wyjściu (stdout), czyli domyślnie na ekranie.
- Przełącznik -n w poleceniu echo sprawia, że na końcu nie jest wysyłany znak nowej linii, natomiast -e włącza interpretację znaków specjalnych (np. \n – nowa linia, \t – tabulacja, \a – dźwięk alertu).
- W systemie Linux każdy proces operuje na trzech podstawowych strumieniach danych powiązanych z deskryptorami:
- **0 (stdin)** – standardowe wejście (domyślnie klawiatura).
- **1 (stdout)** – standardowe wyjście (domyślnie ekran).
- **2 (stderr)** – standardowe wyjście błędów.
- Przełączanie i przekierowywanie strumieni realizuje się za pomocą operatorów: > (zapis z nadpisaniem pliku), >> (dopisanie na końcu pliku) oraz 2> (kierowanie błędów do pliku).

3. Cytowanie i zmienne

- **Cudzysłów ("...")**: zachowuje znaczenie specjalne dla podstawiania wartości zmiennych ($), znaków maskujących (\) oraz odwrotnych apostrofów.
- **Apostrof ('...')**: wyłącza interpretację wszystkich znaków specjalnych, traktując całą zawartość dosłownie jako zwykły łańcuch tekstowy.
- **Odwrotne apostrofy (`...`) lub konstrukcja $(...)**: pozwalają na podstawienie wyniku wykonania polecenia pod zmienną.

- Zmienne definiuje się bez spacji wokół znaku = (np. x="napis"), a odwołuje się do nich, poprzedzając ich nazwę znakiem $ (np. $x).
- **Wybrane zmienne specjalne**:
- $0 – nazwa bieżąco uruchomionego skryptu.
- $1 do $9 – parametry przekazane do skryptu z wiersza poleceń.
- $@ – wszystkie parametry przekazane do skryptu.
- $? – kod powrotu ostatnio wykonywanego polecenia.
- $$ – identyfikator procesu (PID) bieżącej powłoki.
- Zmienne globalne (środowiskowe) udostępniane procesom potomnym definiuje się za pomocą polecenia export zmienna="wartość".

4. Tablice jednowymiarowe

- Deklaracja tablicy polega na przypisaniu listy wartości rozdzielonych spacjami w nawiasach okrągłych: zmienna=(element1 element2 element3).
- Odwołanie do konkretnego elementu następuje poprzez podanie indeksu w nawiasach kwadratowych, np. ${tablica[0]}.
- Symbole @ lub * użyte jako wskaźnik reprezentują wszystkie elementy tablicy jednocześnie (np. ${tablica[*]}).
- Długość wybranego elementu (liczba znaków) uzyskuje się zapisem ${#tablica[0]}, natomiast liczbę wszystkich elementów zwraca ${#tablica[@]}.
- Usunięcie konkretnego elementu tablicy realizuje polecenie unset nazwa_zmiennej[wskaźnik].

5. Instrukcje warunkowe i testowanie

- Instrukcja warunkowa if sprawdza prawdziwość warunku; jeśli jest on spełniony, wykonuje polecenia następujące po słowie then, a całą konstrukcję kończy słowo fi.
- Sprawdzanie warunków odbywa się za pomocą polecenia test lub zapisów w nawiasach kwadratowych [warunek] (warto pamiętać o wymaganych spacjach wewnątrz nawiasów).
- Wybrane operatory testowania plików i wyrażeń:
- -e plik – sprawdza, czy plik istnieje.
- -d wyrażenie – sprawdza, czy obiekt istnieje i jest katalogiem.
- -x plik – sprawdza, czy plik ma nadane prawa do wykonywania.
- = / != – sprawdza równość lub różność ciągów tekstowych.
- Operatory liczbowe: -lt (mniejsze niż), -gt (większe niż), -le (mniejsze lub równe), -ge (większe lub równe).
- Instrukcja wielokrotnego wyboru case pozwala na eleganckie dopasowanie wzorców za pomocą struktury case zmienna in wzorzec) polecenia;; esac.

6. Pętle sterujące

- **Pętla for**: wykonuje blok poleceń dla każdego elementu z zadanej listy.
- **Pętla select**: generuje interaktywne, ponumerowane menu użytkownika na podstawie listy słów.
- **Pętla while**: wykonuje zawarte w niej polecenia tak długo, jak długo sprawdzany warunek pozostaje prawdziwy.
- **Pętla until**: wykonuje polecenia dopóki sprawdzany warunek nie stanie się prawdziwy.

7. Funkcje i wyrażenia arytmetyczne

- Funkcje pozwalają na modularne grupowanie wielokrotnie używanych bloków kodu przy użyciu składni function nazwa_funkcji { ... }.
- Parametry do funkcji przekazuje się bezpośrednio po jej wywołaniu (np. nazwa_funkcji arg1 arg2), a wewnątrz funkcji odczytuje się je standardowo jako $1, $2 itd..
- Obliczenia arytmetyczne na liczbach całkowitych można przeprowadzać za pomocą konstrukcji $((wyrażenie)) lub $[wyrażenie], bądź przy użyciu polecenia let.